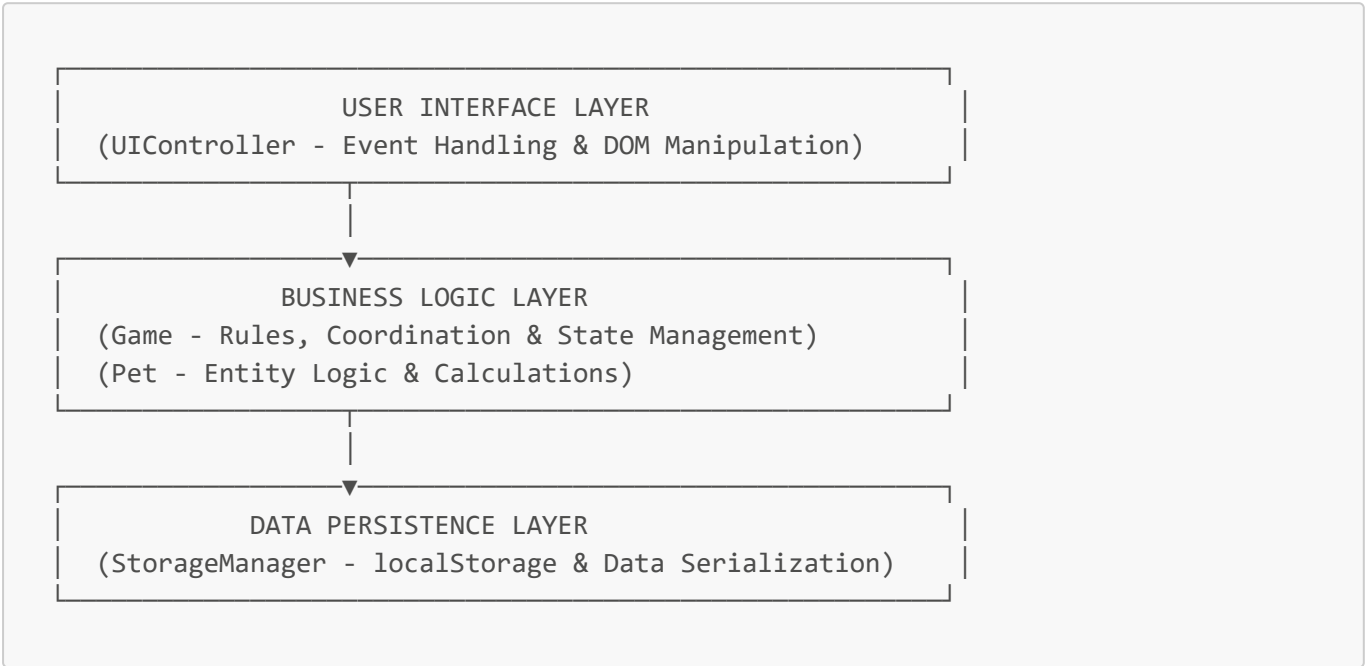


Architecture Documentation

System Architecture Overview

The Virtual Pet Simulator uses a professional three-layer architecture that separates concerns and promotes maintainability.



Class Hierarchy

Game Class (Controller)

Responsibilities:

- Game initialization and setup
- Game loop management
- Rules enforcement
- Action validation
- State management
- Badge checking
- Financial calculations
- Chores/income system coordination

Key Properties:

```
pet: Pet // The pet instance
isRunning: boolean // Game active state
currentBudget: number // Available funds
totalSpent: number // Total expenses
totalEarned: number // Coins earned from chores
```

```
totalChoresDone: number      // Chores completed count
difficulty: string           // 'easy', 'medium', 'hard'
```

Key Methods:

- `initializeGame()` - Set up new game
- `performAction()` - Execute player action
- `doChore()` - Complete a chore for income
- `update()` - Game loop update
- `saveGame()` / `loadGame()` - Persistence

Pet Class (Model)

Responsibilities:

- Pet state management
- Stat calculations
- Emotion determination
- Sickness mechanics
- Level/XP progression
- Action effect application

Key Properties:

```
name: string                // Pet's name
type: string                 // Pet type ('dog', 'cat', 'rabbit', 'bird')
hunger: number               // 0-100
energy: number               // 0-100
happiness: number            // 0-100
cleanliness: number          // 0-100
health: number               // 0-100
level: number                // Current level (unlocks tricks)
xp: number                   // Experience points
```

Key Methods:

- `getEmotion()` - Calculate current emotion
- `updateStats()` - Periodic degradation
- `performAction()` - Apply action effects
- `checkSickness()` - Illness probability

UIController Class (View)

Responsibilities:

- DOM event listening
- UI state updates
- Modal management

- Tab navigation (Care, Finance, Stats, Achievements, Scrapbook, Chat, Reports, Tricks, Chores, Help)
- User input collection
- Feedback display
- Keyboard shortcuts
- API key setup flow

Key Methods:

- `_performAction()` - Handle action button clicks
- `_updateAllUI()` - Refresh all displays
- `_handleFormSubmit()` - Setup form processing
- `_openChatTab()` - Show chat or API key setup
- `_saveApiKey()` - Persist Groq key to localStorage

StorageManager Class (Persistence)**Responsibilities:**

- localStorage operations
- Data serialization
- Game state persistence
- Statistics tracking
- Data export/import

Key Methods:

- `saveGame()` - Persist game state
- `loadGame()` - Restore game state
- `saveExpense()` - Track financial transactions
- `exportData()` / `importData()` - Backup/restore

AICompanion Class (AI Chat)**Responsibilities:**

- Groq API integration (streaming chat)
- Stat-aware system prompt generation
- Persistent chat history and memory
- API key management via localStorage

Key Methods:

- `hasApiKey()` - Check if Groq key is configured
- `setApiKey(key)` - Save key to instance + localStorage
- `chatStream(msg, onToken, onComplete, onError)` - Streaming chat

ReportsCenter Class (Analytics)**Responsibilities:**

- Chart.js chart rendering

- Expense breakdown visualization
- Stat history graphs
- Weekly/daily cost analysis

Scrapbook Class (Memory)

Responsibilities:

- Auto-recording milestone moments
- First-person narrative generation
- Photo/moment storage in localStorage

PersonalityEngine Class (Adaptive AI)

Responsibilities:

- Tracking player behavior patterns
- Adapting pet personality traits over time
- Generating personality-driven responses

HelpCenter Class (FAQ)

Responsibilities:

- 20-item searchable FAQ
- Category-based help topics
- Keyword search across all topics

Validator Class (Input Validation)

Responsibilities:

- Centralized validation for all user inputs
- Pet name, type, budget, difficulty validation
- Groq API key format validation

Data Flow Diagram

Action Execution Flow

```
User Clicks Button
    ↓
UIViewController._performAction()
    ↓
Game.performAction()
    ↓
[Validate]: Check budget, pet state
    ↓
Pet.performAction()
    ↓
Apply stat changes (via ACTION_EFFECTS)
```

```
↓
StorageManager.saveExpense()
↓
Return result to UIController
↓
UIController._showFeedback()
↓
UIController._updateAllUI()
↓
Rendered UI Update
```

Game Loop Update Flow

```
GameLoop Timer (5 seconds)
↓
Game.update()
↓
Pet.updateStats()
↓
[Check]: Sickness probability
↓
Game.checkAchievements()
↓
[Check]: Game over conditions
↓
StorageManager.saveGame()
↓
Return update report
↓
UIController processes report
↓
UI displays changes
```

State Management

Game State Structure

```
{
  pet: {
    name: string,
    type: string,
    hunger: 0-100,
    energy: 0-100,
    happiness: 0-100,
    cleanliness: 0-100,
    health: 0-100,
    isSick: boolean,
    achievements: array,
    // ... more pet properties
```

```
},
gameState: {
  currentBudget: number,
  totalSpent: number,
  initialBudget: number,
  difficulty: string,
  isRunning: boolean,
  sessionStartTime: timestamp
},
achievements: array,
eventLog: array
}
```

Module Dependencies

```
main.js
├── constants.js
├── icons.js
├── PixelSprites.js
├── Validator.js
├── Pet.js
├── StorageManager.js
├── Game.js
│   ├── Pet.js
│   └── StorageManager.js
├── AICompanion.js
│   └── (Groq API via fetch)
├── PersonalityEngine.js
├── Scrapbook.js
├── ReportsCenter.js
│   └── (Chart.js via CDN)
├── HelpCenter.js
└── UIController.js
    ├── Game.js
    ├── AICompanion.js
    ├── ReportsCenter.js
    ├── HelpCenter.js
    └── Scrapbook.js
```

Initialization Sequence

1. Page Load
↓
2. DOM Ready Event
↓
3. Create Game Instance
↓
4. Create UIController Instance
↓

5. Attach Event Listeners
↓
6. Check for Saved Game
↓
7. Either:
 - Load Previous Game OR
 - Show Setup Screen↓
8. Wait for User Input

Storage Schema

localStorage Keys

```
virtual_pet_game_data
└─ Complete game state JSON

virtual_pet_game_data_pet
└─ Pet object serialization

virtual_pet_game_data_config
└─ Game configuration

virtual_pet_game_data_history
└─ Stats history array (last 100 entries)

virtual_pet_game_data_achievements
└─ Unlocked badges array

virtual_pet_game_data_expenses
└─ Expense transactions (last 500 entries)

petpal_groq_key
└─ User-provided Groq API key (set via UI)

petpal_chat_history
└─ AI chat message history

petpal_chat_memory
└─ AI companion extracted memory facts

petpal_scrapbook
└─ Scrapbook milestone entries
```

Performance Considerations

Optimization Techniques

1. **Element Caching** - DOM elements cached at initialization
2. **Batch Updates** - Multiple UI updates in single render

3. **Event Delegation** - Single event listener for multiple elements
4. **Storage Limits** - History and expenses limited to prevent bloat
5. **Interval-Based Updates** - 5-second update intervals (not continuous)

Memory Management

- Pet object: ~2KB
- Game object: ~1KB
- UI state: ~0.5KB
- Local Storage: ~1-5MB total
- Typical Memory: 5-10MB during gameplay

Error Handling

Try-Catch Blocks

All critical operations wrapped in try-catch:

- Game initialization
- localStorage operations
- Pet actions
- Achievement checking

Validation Layers

1. **Input Validation** - Form inputs checked before submission
2. **State Validation** - Pet state checked before actions
3. **Financial Validation** - Budget checked before purchases
4. **Logical Validation** - Game rules enforced

Error Recovery

- Graceful failures with user feedback
- Fallback states for corrupted saves
- Console logging for debugging
- DEBUG utilities for problem diagnosis

Extensibility Points

Easy Enhancements

1. **New Pet Types** - Add to PET_TYPES in constants.js
2. **New Actions** - Add to ACTION_COSTS and ACTION_EFFECTS
3. **New Achievements** - Add to ACHIEVEMENTS object
4. **New Stats** - Extend stat bar system
5. **New Emotions** - Add to EMOTIONS object

Moderate Enhancements

1. **Multiple pets** - Refactor to pet collection

2. **Levels/Evolution** - Add growth system to Pet
3. **Mini-games** - New game modes in Game class
4. **Leaderboards** - Additional storage system
5. **Advanced Analytics** - Enhanced stats tracking

Major Enhancements

1. **Backend API** - Replace localStorage with server
 2. **Multiplayer** - Real-time synchronization
 3. **Mobile App** - React Native/Flutter port
 4. **Progressive Web App** - Service worker integration
 5. **Advanced Graphics** - Canvas/WebGL rendering
-

Architecture Version: 1.0

Last Updated: May 2026

Status: Production Ready